IMPLEMENTASI DATA HAZARDS PADA INSTRUCTION-LEVEL PARALLELISM

Studi RAW, WAR, dan WAW dengan Dependency Graph

Mata Kuliah	Organisasi dan Arsitektur Komputer
Topik	Mitigasi Data Hazards pada ILP
Jenis Pengerjaan	Tugas Mandiri
Nama Mahasiswa	Isi nama sendiri sebelum dikumpulkan
NIM	Isi NIM sendiri
Repository GitHub	Tambahkan URL setelah repository dibuat

Disusun sebagai implementasi dan analisis mandiri berdasarkan instruksi tugas dan materi kuliah yang diberikan.

1. Pendahuluan

Instruction-Level Parallelism (ILP) adalah kemampuan prosesor atau compiler untuk menjalankan beberapa instruksi secara tumpang tindih atau paralel ketika dependensi instruksi memungkinkan. Data hazard muncul ketika urutan baca/tulis register memengaruhi kebenaran hasil apabila instruksi dijadwalkan tanpa memperhatikan dependensi.

Laporan ini mengimplementasikan tiga pola dependensi: Read After Write (RAW), Write After Read (WAR), dan Write After Write (WAW). Setiap contoh memakai dua instruksi agar hubungan antarinstruksi terlihat langsung.

2. Metode Implementasi

Setiap instruksi direpresentasikan oleh himpunan register yang dibaca (READ) dan ditulis (WRITE). Program membandingkan pasangan instruksi menurut urutan program. Jika kondisi hazard terpenuhi, dibuat directed edge dari instruksi lebih awal ke instruksi berikutnya.

Catatan: ini adalah simulator edukatif sederhana yang memeriksa register pada contoh instruksi. Ini bukan model siklus pipeline lengkap dan tidak mengklaim mereproduksi seluruh fitur internal notebook Colab dosen.

3. Implementasi RAW — Read After Write

RAW terjadi saat instruksi yang lebih muda membaca nilai yang ditulis oleh instruksi lebih tua. Ini adalah true data dependence karena instruksi kedua membutuhkan hasil instruksi pertama.

Instruksi:

```
I1: ADD R1, R2, R3    ; R1 = R2 + R3\nI2: ADD R4, R1, R5    ; R4 = R1 + R5
```

Dependency graph:

```
[I1: menulis R1]  ■■ RAW (R1) ■■>  [I2: membaca R1]
```

Analisis: I2 tidak boleh membaca R1 sebelum I1 menghasilkan nilai baru R1. Mengubah nama register saja tidak menghilangkan kebutuhan nilai tersebut. Forwarding dapat membantu mengurangi stall, tetapi dependensi logis tetap ada.

4. Implementasi WAR — Write After Read

WAR terjadi ketika instruksi lebih tua membaca register yang kemudian ditulis instruksi lebih muda. Pembacaan I1 harus terjadi sebelum penulisan baru I2 menimpa register yang sama.

Instruksi:

```
I1: ADD R1, R2, R3    ; membaca R2\nI2: ADD R2, R4, R5    ; menulis R2
```

Dependency graph:

```
[I1: membaca R2]  ■■ WAR (R2) ■■>  [I2: menulis R2]
```

Analisis: WAR merupakan name dependence (dependensi nama) pada contoh ini, bukan kebutuhan nilai hasil I1. Register renaming dapat memetakan penulisan I2 ke register fisik lain sehingga konflik nama hilang, selama pemetaan register tetap benar.

5. Implementasi WAW — Write After Write

WAW terjadi saat dua instruksi menulis register tujuan yang sama. Jika penulisan diselesaikan dalam urutan yang salah, nilai akhir register dapat berasal dari instruksi yang keliru.

Instruksi:

```
I1: ADD R1, R2, R3    ; menulis R1\nI2: ADD R1, R4, R5    ; menulis R1
```

Dependency graph:

```
[I1: menulis R1]  ■■ WAW (R1) ■■>  [I2: menulis R1]
```

Analisis: sesuai urutan program, hasil akhir R1 harus berasal dari I2. WAW merupakan name dependence pada contoh ini dan dapat diatasi dengan register renaming sehingga kedua instruksi menulis ke register fisik berbeda dan hasil dipetakan sesuai urutan arsitektural.

6. Kode Program dan Cara Menjalankan

Kode Python tersedia pada file ilp_hazards.py dan notebook ILP_Data_Hazards_Mandiri.ipynb. Jalankan di Python 3 atau unggah notebook ke Google Colab, lalu pilih Runtime → Run all.

```
python ilp_hazards.py
```

Hasil inti yang diharapkan:

```
RAW : I1 ---- RAW (R1) ----> I2\nWAR : I1 ---- WAR (R2) ----> I2\nWAW : I1 ---- WAW (R1) ----> I2
```

7. Perbandingan

Jenis	Pola	Kategori	Makna urutan
RAW	I1 write R1; I2 read R1	True data dependence	I2 membutuhkan nilai hasil I1
WAR	I1 read R2; I2 write R2	Name dependence	I1 membaca sebelum R2 ditimpa
WAW	I1 write R1; I2 write R1	Name dependence	Urutan penulisan menjaga nilai akhir

8. Kesimpulan

RAW, WAR, dan WAW membatasi penjadwalan instruksi, tetapi alasannya berbeda. RAW menjaga aliran nilai yang benar dan merupakan true data dependence. WAR dan WAW pada contoh ini merupakan name dependence yang dapat dikurangi melalui register renaming. Dependency graph membantu menunjukkan urutan relatif yang perlu dipertahankan agar optimasi ILP tidak mengubah hasil program.

9. GitHub dan Reproduksibilitas

URL repository belum dapat dicantumkan sebagai tautan nyata karena repository harus dibuat dan diunggah melalui akun GitHub pemilik tugas. Buat repository bernama **ilp-data-hazards-mandiri**, lalu masukkan URL repository pada sampul. Petunjuk resmi: https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github.

Contoh perintah setelah repository kosong dibuat:

```
git init\ngit add .\ngit commit -m "Implementasi awal RAW, WAR, WAW"\ngit branch -M main\ngit remote add ori
```

Ganti USERNAME dengan username GitHub sendiri. Lakukan commit lanjutan setelah menguji program dan memperbaiki dokumentasi agar riwayat commit mencerminkan pekerjaan yang dilakukan.

Daftar Pustaka

1. Materi kuliah, “Paralelisme CPU: Mitigasi Data Hazards pada ILP”, Organisasi dan Arsitektur Komputer, Informatika ITSB, 15 Oktober 2026.

2. GitHub Docs, “Adding locally hosted code to GitHub”, https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github.